Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:- ShopEZ-E-commerce-Application**

- **Team Members:- INDIVIDUAL**

2. Project Overview

- **Purpose**: To provide a reliable, user-friendly MERN stack marketplace that connects buyers and sellers, allowing users to discover and purchase products online while giving small sellers a robust digital storefront.
- **Features:** Comprehensive product catalog, secure user authentication, shopping cart management, seamless checkout flow, and an admin/seller dashboard.

3. Architecture

- **Frontend:** Built using React.js for a responsive, dynamic single-page application (SPA) UI.
- **Backend:** Powered by Node.js and Express.js to handle API routing, business logic, and user sessions.
- **Database:** A local instance of MongoDB is used to store Users, Products, and Orders, utilizing Mongoose ODM for schema definitions and data interactions.

4. Setup Instructions

- **Prerequisites:** Node.js and MongoDB must be installed locally.
- **Installation:**
- Clone the repository to your local machine.
- Navigate to the `server` directory and run `npm install` to install dependencies.
- Navigate to the `client` directory and run `npm install` to install dependencies.
- Set up environment variables (`.env`) for the local MongoDB URI and JWT Secret.

5. Folder Structure

- **Client:** Contains the React application, organized into components (reusable UI elements), pages (Home, Cart, Checkout), and assets.
- **Server:** Contains the Node.js application, organized into models (Mongoose schemas), controllers (business logic), routes (API endpoints), and middleware.

6. Running the Application

- **Frontend:** Run `npm start` in the client directory to launch the React development interface.
- **Backend:**Run `npm start` in the server directory to boot the local Express server.

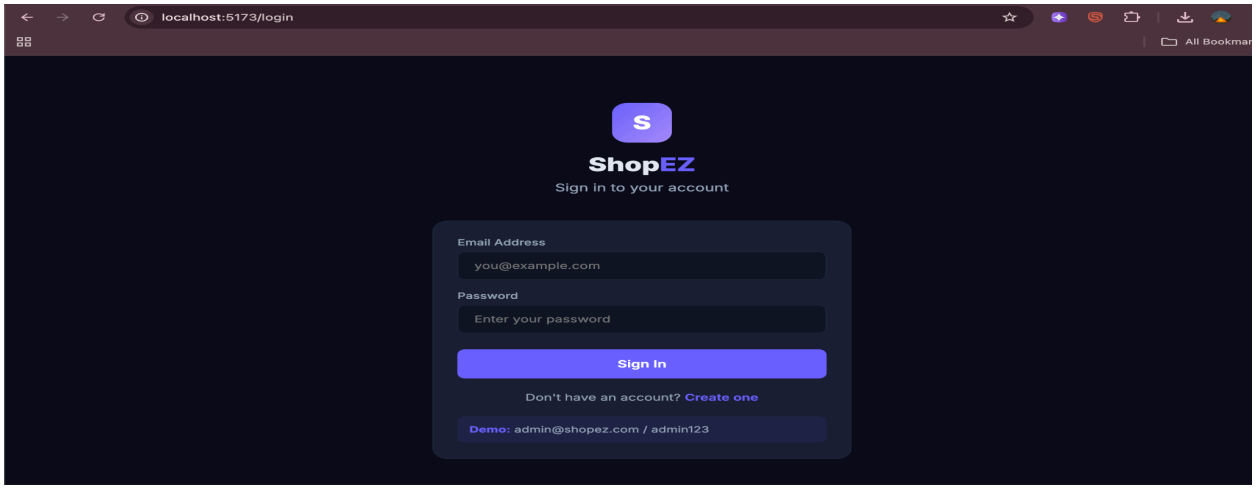
7. API Documentation

- `POST /api/auth/register`: Registers a new user. Parameters: name, email, password.
- `POST /api/auth/login`: Authenticates a user. Returns a JWT.
- `GET /api/products`: Fetches the entire product catalog.

8. Authentication

- Authentication and route-level authorization are handled using JSON Web Tokens (JWT).
- Passwords are encrypted before being saved to the database using `bcrypt.js`.

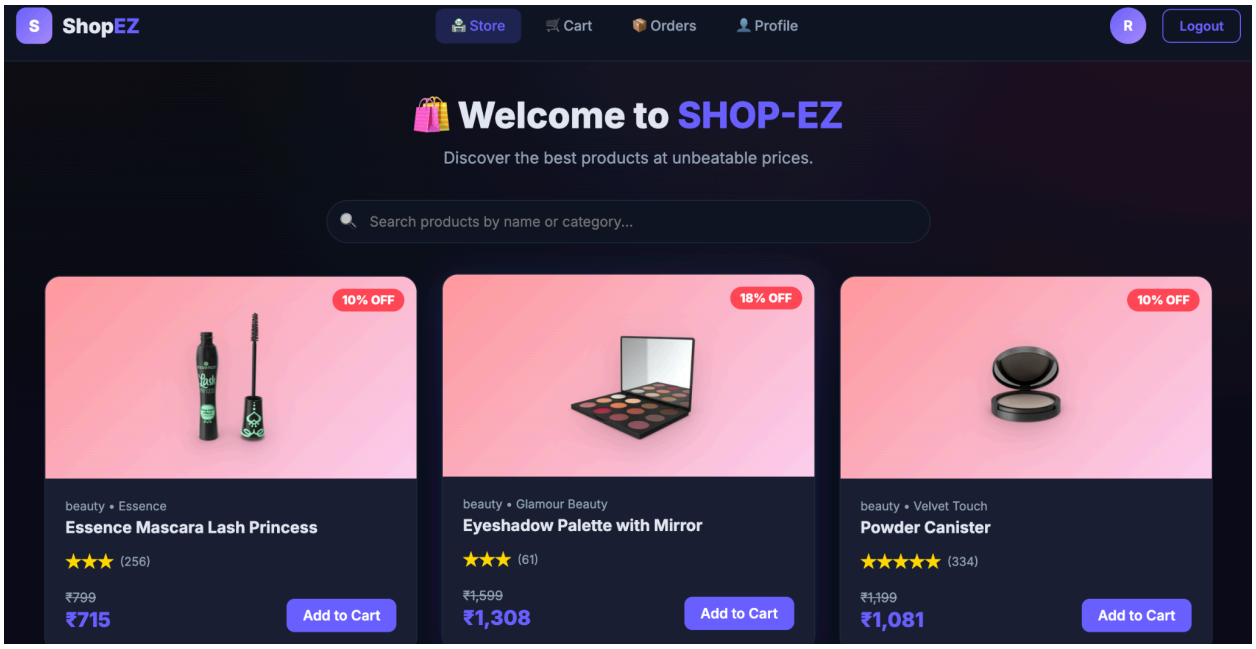
9. User Interface



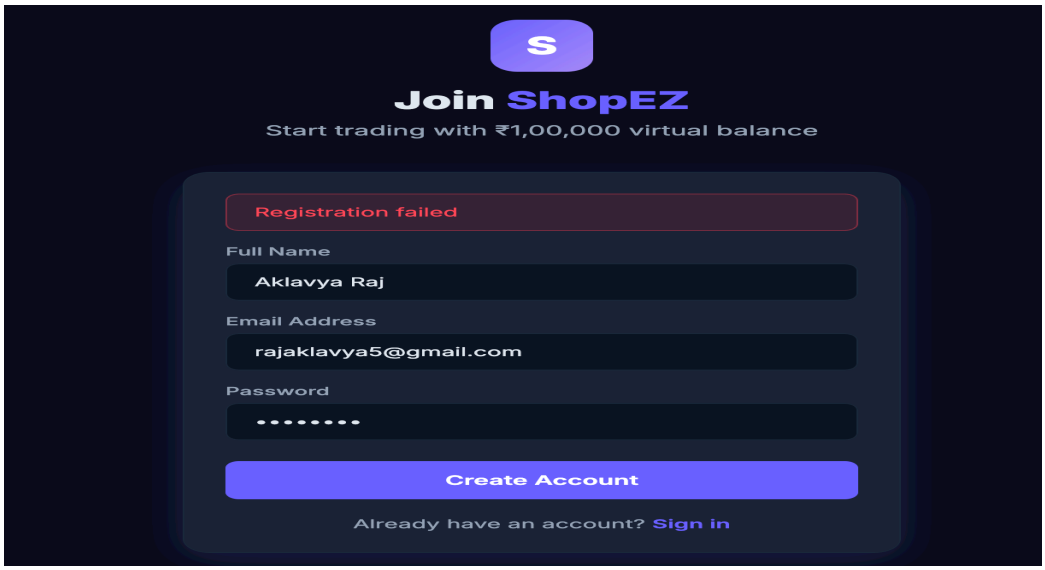
10. Testing

- **Strategy:** User Acceptance Testing (UAT) is conducted to ensure all positive and negative scenarios (like login failures or cart updates) function correctly before final sign-off.

11. Screenshots or Demo



12. Known Issues



13. Future Enhancements

- Migration from a local localhost environment to cloud deployment, implementing a secure third-party payment gateway, and adding an AI-driven product recommendation engine.